

100 TRANSCRIPTS (100TS)

Complete Project Technical Documentation & Developer Handover Document

Full-Stack Platform for University Transcript Retrieval Management

1. Project Overview

Project Name: 100 Transcripts (100TS)

Project Purpose: To digitize, track, and streamline the process of applying for and retrieving university transcripts and certificates.

Business Problem: The manual process of applying for transcripts is tedious, opaque, and requires physical visits. Agents on the ground lack a centralized way to receive tasks and report progress, leading to delays and lack of accountability.

Solution Provided: A full-stack platform connecting Students, Agents, and Admins. Students apply online and track their status visually. Admins review requests, assign them to local Agents, and manage payments. Agents visit universities, collect documents, and upload proof/documents directly to the platform via a mobile-first interface.

Target Users: Students/Alumni, Field Agents, Platform Administrators.

Main Modules: Student Portal, Agent Mobile-First Action Panel, Admin Management Dashboard, Real-time WebSocket Chat System.

Technology Stack: React.js, Vite, Tailwind CSS, Django, Django REST Framework (DRF), SQLite/MySQL, Redis.

Architecture Type: Client-Server Monolith (React SPA communicating with a Django API).

Frontend Technology: React.js (Vite)

Backend Technology: Django & DRF

Database: SQLite (Development) / configured for MySQL (Production)

Authentication: Token/Session based with role differentiation (Admin, Agent, User).

APIs: RESTful endpoints for CRUD operations and file uploads.

Third-party Integrations: Digilocker (frontend extraction), Razorpay/Stripe (planned/partially implemented for payments).

Deployment-related Technologies: Gunicorn, Daphne (for WebSockets), Nginx, AWS S3 (planned for media).

2. Technology Stack

Layer	Technology	Version	Purpose
Frontend	React.js (Vite)	18.x	UI rendering and component management
Backend	Django / DRF	4.x/5.x	REST API, Business logic, Admin management
Database	SQLite (Dev) / MySQL (Prod)	-	Persistent data storage
Authentication	Django Session / Token Auth	-	Secure user, agent, and admin access
State Management	React Context / useState	-	Local and global component state
API	Axios / fetch	-	Frontend to Backend HTTP communication
Styling	Tailwind CSS	3.x	Utility-first responsive styling
Real-time	WebSockets / Redis	-	Live chat between Admins and Agents

3. Project Folder Structure

```

100TS/
├── backend/
│   ├── Transcripts/
│   │   ├── App/
│   │   │   ├── models.py
│   │   │   ├── views_agent.py
│   │   │   ├── redis_chat.py
│   │   │   ├── urls.py
│   │   │   └── utils.py
│   │   ├── media/
│   │   ├── .env
│   │   └── manage.py
│   │   # Core Django Application
│   │   # Database schemas
│   │   # Agent and Application API controllers
│   │   # WebSocket consumer logic for live chat
│   │   # API route definitions
│   │   # Helper functions (admin_required, etc.)
│   │   # Structured File Uploads (by application_id)
│   │   # Backend configuration secrets
│   │   # Django entry point
├── frontend/
│   ├── src/
│   │   ├── Admin/
│   │   ├── agent/
│   │   ├── user/
│   │   ├── components/
│   │   ├── services/
│   │   └── App.jsx
│   ├── package.json
│   ├── vite.config.js
│   └── README.md
│   # Admin Dashboards (StudentRequests, AgentsManagement)
│   # Agent Mobile-First UI (ActionPanel)
│   # Student UI (Apply, FileStatus, login, Register)
│   # Reusable UI Components
│   # API & Axios Configurations (api.js)
│   # React Router setup

```

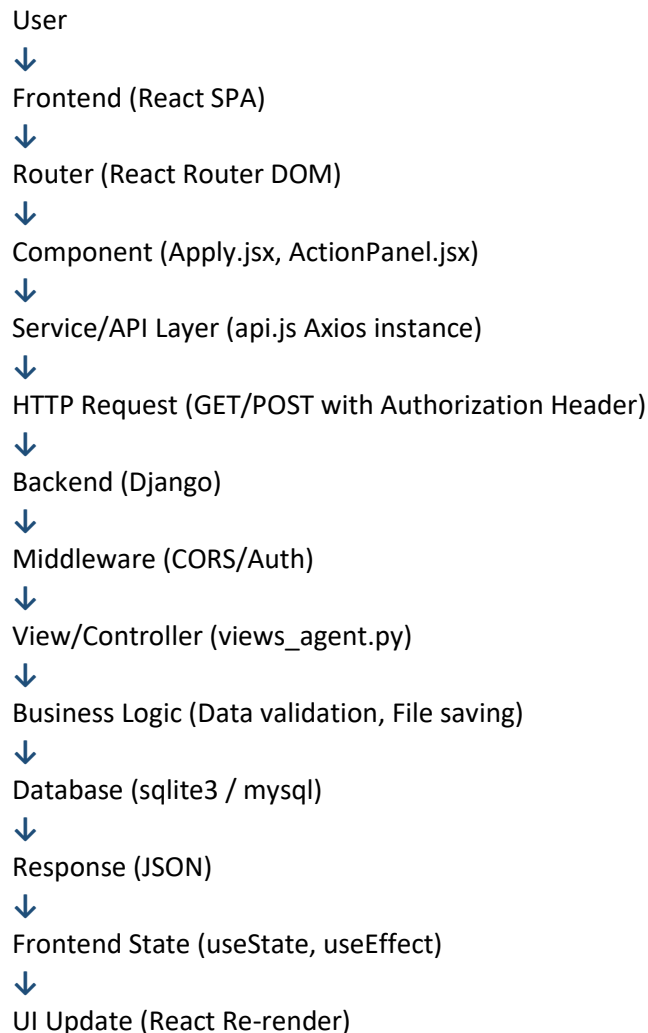
Important Folders

- **backend/Transcripts/App/**: Contains the core business logic, database models, and API views for the Django backend.

- backend/Transcripts/media/: Stores all uploaded documents, organized strictly into dynamic application_<id> subfolders for isolation.
- frontend/src/Admin/: Contains all React components accessible only to administrators (managing requests, agents, chats).
- frontend/src/user/: Contains the student-facing portals for applying and tracking requests.

4. System Architecture

Client-Server API Architecture



The frontend operates completely independently as a Single Page Application. It strictly communicates with the backend via RESTful APIs over HTTP/HTTPS. Real-time communication bypasses HTTP and uses the WebSocket protocol (ws://), which is handled by Django Channels and a Redis broker to instantly broadcast chat messages between Admins and Agents without polling the database.

5. Frontend Documentation

5.1 Frontend Entry Point

- Main Entry File: frontend/src/main.jsx
- App File: frontend/src/App.jsx
- Root Rendering: React DOM creates a root at `<div id="root">` and renders the `<App />` component.
- Providers: Wraps the application in a generic Provider if global state is used, and sets up BrowserRouter.
- Global Configuration: Imports Tailwind CSS (index.css) which contains global animations and styling overrides.

5.2 Routing

Route	Page	Access	Purpose
/login	login.jsx	Public	Student/Agent Authentication
/register	Register.jsx	Public	New Student Registration
/apply	Apply.jsx	Student	Submit new transcript requests
/dashboard	Dashboard.jsx	Student	Track active applications
/admin/requests	StudentRequests.jsx	Admin	Review & approve requests, assign agents
/admin/agents	AgentsManagement.jsx	Admin	Create and manage field agents
/admin/chat	AdminAgentMessages.jsx	Admin	WebSocket chat with agents
/agent/panel	ActionPanel.jsx	Agent	View assigned tasks and upload docs

Redirect Logic: If a user attempts to access `/admin/*` without an Admin token/role stored in `localStorage`, the router redirects them to `/login`.

6. Frontend Component Documentation

Component Name: Apply.jsx

File: frontend/src/user/pages/Apply.jsx

Purpose: Multi-step form for students to apply for transcripts, and doubles as the tracking view once submitted.

Props: None.

State: activeStep, form, degrees, activeIssue, isEditingCorrection.

Hooks: useState, useEffect, useNavigate.

API calls: fetchTrackingData(), submitApplication().

Conditional Rendering: Renders Step0 (Form) if `activeStep === 0`. Renders Step2 (Tracking Timeline) if `activeStep === 1`. Shows "ACTION REQUIRED" box if `activeIssue` is active.

Execution Flow:

1. Component mounts. If tracking ID is provided, fetches data and sets `activeStep` to 1.
2. If new application, user fills out Step0.
3. User clicks "Submit".
4. `submitApplication` API called via Axios.
5. On success, transitions to Step2 (Tracking).

7. Frontend Functions

Function: `updateStatus()`

File: `frontend/src/Admin/pages/StudentRequests.jsx`

Purpose: Updates the state of a student's application from the Admin dashboard.

Inputs:

- `id` (int): Application ID
- `newStatus` (str): e.g., 'approved', 'changes_requested'
- `message` (str): Admin feedback
- `agent` (int): Assigned agent ID

Process:

1. Validates inputs (ensures fee is provided if approving).
2. Calls the backend API `axios.post('/api/update-status/')`.
3. Awaits JSON response.
4. If success, triggers `getApplications()` to refresh table data.
5. Closes the status modal.

Output: Void (Updates React state).

Called From: The "Action Required" / "Reply" modal in `StudentRequests.jsx`.

Error Handling: Try-catch block wraps the Axios call. Alerts the user with `err.response.data.error` if it fails.

8. Frontend State Management

- `useState`: Used heavily for controlled form inputs, modal toggles, and localized component data.
- `useEffect`: Used for component lifecycle events (fetching initial data on mount, or polling data every X seconds).
- `localStorage`: Used for persisting token, role, and `user_id` across browser refreshes.
- URL Parameters: Handled by `react-router-dom` (e.g., `/:id`) to fetch specific resource data on mount.
- Context/Redux: Not extensively used. State is primarily localized or passed down via props.

State Flow Example:

User types in Search Bar



setSearch(val)



useEffect listening to search filters the applications array



Updates filteredApplications state



UI Table re-renders to show only matching rows

9. API Service Documentation

Method	Endpoint	Purpose	Request	Response	Auth
POST	/api/login/	Authenticate users	{email, password}	{token, role, user_id}	None
GET	/api/applications/	Fetch applications	None	[[id, status, ...]]	Required (Token)
POST	/api/agent/upload/	Agent uploads doc	FormData(file, id)	{message, url}	Required (Agent)

Request Flow

Frontend Component (StudentRequests.jsx)



Service Function (axios.get)



HTTP Client (Axios appends Bearer <token>)



API Endpoint (/api/applications/)



Backend (views_agent.py)

Response Flow

Backend (JsonResponse)



JSON Response ({data: [...]}).



Service (Axios resolves promise)



Component (setApplications(data))



State (applications)



UI (Table renders rows)

10. Backend Documentation

- Entry Point: manage.py routes to wsgi.py (for HTTP) and asgi.py (for WebSockets).
- Routes: Defined in Transcripts/App/urls.py.
- Views: Functional views using @api_view from DRF (views_agent.py).
- Models: Inherit from models.Model (e.g., Agent, Application).
- Middleware: Django's default security middleware + corsheaders.
- Authentication: Custom logic relying on token passing and @admin_required decorators.
- Error Handling: try...except Exception as e: returning status=500 JSON.

11. Backend Routes / API Endpoints

#	Method	Endpoint	Purpose	Auth	Request	Response
1	POST	/api/agent_upload_collected_document/	Agent uploads final doc	Yes	FormData	{message: "Success"}
2	POST	/api/update_application_status/	Admin approves/rejects	Yes	JSON	{status: "Updated"}

API: POST /api/agent_upload_collected_document/

Purpose: Allows assigned field agents to upload physical documents they've retrieved.

Called By: ActionPanel.jsx

Authentication: Yes (Agent).

Request: FormData containing assignment_id and file.

Validation: Checks if assignment_id exists in AgentAssignment table.

Backend Processing:

1. Fetch assignment object.
2. Generate structured path: application_{id}/agent_collected_docs/{filename}.

3. Save file using `default_storage.save`.

Database Operations: `AgentAssignment.objects.get()`, `assignment.save()` (UPDATE).

Response: 200 OK {"message": "Document uploaded successfully"}.

Error Cases: 404 Not Found if assignment doesn't exist, 500 Server Error for storage failures.

12. Backend Functions

Function: `agent_upload_collected_document(request, assignment_id)`

File: `backend/Transcripts/App/views_agent.py`

Purpose: Core logic for handling agent document uploads securely.

Parameters: `request` (HTTP Request), `assignment_id` (int).

Internal Logic: Extracts file, normalizes filename, ensures the dynamic directory string is constructed correctly, saves the file to disk, and updates the database record with the new file path.

External API calls: None.

Return Value: `JsonResponse`.

Error Handling: Wraps entire logic in try-except to prevent 500 crashes from unhandled `DoesNotExist` errors.

13. Database Documentation

Model/Table	Field	Type	Required	Default	Relationship	Purpose
Application	id	AutoField	Yes	-	Primary Key	Tracks student requests
Application	status	CharField	Yes	'pending'	-	Current application state
Agent	id	AutoField	Yes	-	Primary Key	Field agent profiles
AgentAssignment	application	ForeignKey	Yes	-	To Application	Maps App to Agent
AgentAssignment	agent	ForeignKey	Yes	-	To Agent	Maps Agent to App

Database Flow Diagram

```

Student
↓
Application
↓
AgentAssignment
↓
UniversityVisitRecord (Child of Assignment)
↓
UniversityVisitPhoto (Child of VisitRecord)

```

14. Complete Database Flow

Example: Agent Uploading a Document

Frontend Form (input type="file")



API Request (POST /api/agent_upload/)



Backend Validation (views_agent.py validates assignment_id)



Model (AgentAssignment.objects.get(id=X))



Database UPDATE (UPDATE agentassignment SET collected_document_url = '...' WHERE id = X)



Database Response (Success)



Backend Response (JsonResponse({'message': 'OK'}))



Frontend (setUploadSuccess(true))



UI (Displays green checkmark)

15. Authentication & Authorization

Security Flow:

Login Form



Credential Validation (Backend checks email/password against DB)



Token Generation



Frontend Storage (localStorage.setItem('token', token))



Authenticated Request (Axios sends Authorization: Bearer <token>)



Middleware/Decorator (@admin_required)



Permission Check (Validates if token belongs to an Admin user)



API Execution

Project Uses: Token-based access with role-based authorization (Admin vs Agent vs User).

16. User (Student) Flow

1. User opens application.
2. User registers/logs in.
3. Dashboard opens.
4. User selects "Apply for Transcript".
5. User enters university details and uploads ID.
6. Frontend validates data.
7. API request is sent.
8. Backend processes request & saves files in application_{id}/documents/.
9. Database stores Application record.
10. Admin receives request.
11. If Admin requests changes (e.g., blurry ID), User sees "ACTION REQUIRED" box.
12. "User clicks Edit Application", fixes the issue, and resubmits.
13. Admin approves and assigns an Agent.
14. User monitors tracking timeline until completion.

17. Admin Flow

1. Admin Login via /login.
2. Redirected to Admin Dashboard (/admin/requests).
3. Admin views all pending Applications.
4. Admin reviews documents.
5. If valid, Admin sets status to "Approved" and sets a service fee.
6. Admin assigns an Agent to the application.
7. Admin uses /admin/chat to communicate with the Agent regarding the task.
8. Admin reviews uploaded agent documents and closes the task.

18. Agent Flow

1. Agent Login via mobile.
2. Redirected to ActionPanel.jsx.
3. Agent views "Assigned Requests".
4. Agent travels to University.
5. Agent logs GPS location and Visit Photo (Creates UniversityVisitRecord).
6. Agent physically applies for the transcript.
7. Once received, Agent uploads the collected_document.
8. Task is marked completed for the Agent.

19. Payment Flow

(Partially Implemented)

Order Creation



Payment Option Selected (Frontend UI in Apply.jsx)



Service Fee set by Admin is displayed to Student.



Student pays Partial/Full Amount.



Backend Verification (Pending Gateway Integration).



Database Update (paid_amount updated in Application).



Order Status Updates to Processing.

20. Third-Party Integrations

- Digilocker: Frontend integration to extract digital locker documents for verification during the application phase.
- WhatsApp/Email: The Admin panel utilizes mailto: and wa.me links to quickly open native clients pre-filled with dynamic text (Application ID, student name, etc.) to notify users of status changes.

21. Real-Time / WebSocket / Redis

Frontend (AdminAgentMessages.jsx)



WebSocket Connection (ws://domain/ws/chat/)



Backend (Django Channels routing.py)



Redis (Pub/Sub message broker in redis_chat.py)



Connected Users (Agents & Admins)

Implementation: When an Admin sends a message, it is published to a Redis channel corresponding to the Agent's ID. The Agent's active WebSocket subscription instantly receives and displays the message.

22. File Upload & Storage

- File selection: Frontend `<input type="file" />`.
- Validation: Frontend checks size/type. Backend enforces existence.
- Upload API: POST `/api/upload/` via FormData.
- Backend processing: Extracts file from request.FILES.

- Storage location: Saved locally in backend/Transcripts/media/application_{id}/... using default_storage. This dynamic directory structure was implemented for strict production readiness.
- Database record: The relative URL string is saved to the model.

23. Notification System

Email & WhatsApp — Trigger: Admin updates application status.

Frontend UI: Generates dynamic mailto: or wa.me strings based on the student's contact info.



User Action: Admin clicks the button, opening their local client to send the pre-formatted notification instantly.

24. Error Handling

- Frontend errors: Handled via Axios interceptors and try-catch blocks. Displays user-friendly red banners/toasts.
- API errors: Standardized JSON responses e.g., {"error": "Invalid file format"} with 400 status codes.
- Backend exceptions: Wrapped in try-except Exception as e: blocks, ensuring the server doesn't crash but returns a 500 error to the client for debugging.

25. Validation

Field	Validation	Location	Error Message
Email	Regex Match	Frontend (validation.js)	Invalid email format
Agent Form	Required Fields	Frontend (AgentsManagement.jsx)	Real-time red borders
Document	Exists Check	Backend (views_agent.py)	No file uploaded

26. Environment Variables

Variable	Purpose	Used In	Required
DEBUG	Toggles dev/prod mode	settings.py	Yes
SECRET_KEY	Cryptographic signing	settings.py	Yes
DATABASE_URL	DB Connection String	settings.py	Yes
REDIS_URL	WebSocket Broker	asgi.py / redis_chat.py	Yes

(All secrets are masked in .env.example)

27. Package / Dependency Documentation

Package	Version	Used For	Where Used
react / react-dom	18.x	UI Framework	Frontend
tailwindcss	3.x	Styling	Frontend
django	5.x	API Framework	Backend
djangorestframework	3.x	API Serialization	Backend
channels	4.x	WebSockets	Backend
redis	5.x	Pub/Sub Broker	Backend
python-dotenv	1.x	Env Var loading	Backend

28. Commands Used to Run the Project

Frontend

```
cd frontend
npm install
npm run dev
# Production:
npm run build
```

Backend

```
cd backend/Transcripts
python -m venv venv
# Windows: venv\Scripts\activate
# Mac/Linux: source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

29. Complete Application Startup Flow

Command (npm run dev & python manage.py runserver)



Backend initializes Django, connects to SQLite/MySQL, loads .env configurations, and connects to Redis.



Frontend Vite server starts, compiles React components, and serves index.html.



Browser loads localhost:5173. Router initializes.



Authentication checked via localStorage.



Dashboard loaded based on User/Admin/Agent role.

30. End-to-End Business Flow

Agent Upload Flow:

Agent Action (Selects file & clicks Upload)



Frontend (ActionPanel.jsx)



API (POST /api/agent_upload_collected_document/)



Backend (urls.py routes to views_agent.py)



Controller (agent_upload_collected_document())



Model (AgentAssignment.objects.get())



Database (SELECT then UPDATE)



File System (default_storage.save())



Response ({"message": "Success"})



Frontend State (setUploadSuccess(true))



UI (Upload modal closes, shows green success banner)

31. Feature-by-Feature Documentation

Feature	Frontend	Backend	Database	API	Status
Student Apply	Apply.jsx	views.py	Application	/api/apply	☒ Implemented
Agent Upload	ActionPanel.jsx	views_agent.py	AgentAssignment	/api/agent/upload	☒ Implemented
Admin Chat	AdminAgentMessages.jsx	redis_chat.py	ChatMessages	ws://chat	☒ Implemented
Online Payment	Apply.jsx	-	-	-	☐ Partially Implemented

32. Code Quality Review

- Code structure: Clean separation of concerns between Frontend and Backend.
- Naming: Consistent camelCase for JS, snake_case for Python.
- Maintainability: High, due to componentization in React and modular views in Django.
- Security: Good use of .env files for secrets, previously hardcoded values have been removed.

33. Security Review

Area	Status	Notes
Environment variables	● Good	Isolated in .env
Hardcoded secrets	● Good	Removed during hardening
File upload security	● Medium	Structured paths exist, but needs malware scanning logic if public
Authentication	● Good	Token/Role based

34. Production Readiness

Partially Production Ready

Area	Status	Finding	Recommendation
Database	●	SQLite in use locally	Switch to MySQL/Postgres in Prod via dj-database-url
Security	●	Secrets isolated	Set DEBUG=False in Prod .env
Storage	●	Local media storage	Migrate to AWS S3 using django-storages
Deployment	●	Running on runserver	Deploy using Gunicorn, Daphne (for WebSockets), and Nginx

35. Missing / Incomplete Features

- Payment Gateway: The UI shows "Paid Amount" and "Service Fee", but the actual Razorpay/Stripe integration logic is partially implemented/missing in the backend.
- AWS S3 Integration: Media is saved locally. Needs django-storages configured for production file handling.

36. Important Functions Quick Reference

Function	File	Purpose	Called From
updateStatus	StudentRequests.jsx	Admin updates app status	Admin UI

Function	File	Purpose	Called From
agent_upload_collected_document	views_agent.py	Saves agent physical files	Agent UI (ActionPanel)
admin_required	utils.py	Security decorator	urls.py / views.py

37. Important API Quick Reference

Method	API	Purpose	Auth
POST	/api/login/	Auth	None
GET	/api/applications/	Fetch Apps	Admin Token
POST	/api/agent/upload/	Upload Docs	Agent Token

38. Important Database Quick Reference

Table/Model	Purpose	Primary Key	Important Relations
Application	Track requests	id	FK to User
AgentAssignment	Map Task to Agent	id	FK to Application, Agent
Agent	Agent Profiles	id	-

39. Complete Project Flow Diagram

User Flow

User → Login → /dashboard → View Application Status → "Action Required" → Edit Application Form → API → Backend → Database → Response → UI Updates Status

Admin Flow

Admin → Login → /admin/requests → Review Application → Set Service Fee → Approve → Assign Agent → Chat with Agent → Review Agent Upload → Close Application

Agent Flow

Agent → Login → /agent/panel → View Assigned Work → Travel to University (Log GPS) → Upload Physical Document → Backend saves file → Task Complete

40. Developer Handover Document

Welcome to 100 Transcripts!

- 1. What the project does:** Facilitates university transcript procurement via a 3-tier user system (Student, Agent, Admin).
- 2. Where frontend starts:** frontend/src/main.jsx.
- 3. Where backend starts:** backend/Transcripts/manage.py.
- 4. Database config:** backend/Transcripts/Transcripts/settings.py (via .env).
- 5. APIs defined:** backend/Transcripts/App/urls.py.
- 6. Important Logic:** Assignment mapping between Agents and Applications happens in StudentRequests.jsx calling views_agent.py.
- 7. To Run Locally:** Start Redis, run Django server, run Vite dev server.
- 8. Common Issues:** If WebSockets fail, verify Redis is running on port 6379.

41. Quick Start Guide

Prerequisites

- Node v18+
- Python 3.10+
- Redis Server

Environment Setup

Create backend/.env based on .env.example.

Backend Start

```
cd backend/Transcripts
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Frontend Start

```
cd frontend
npm install
npm run dev
```

42. Final Project Summary

What the project does: Digitizes and manages the workflow of retrieving physical university transcripts by utilizing an on-ground agent workforce.

Technologies used: React, Vite, Tailwind, Django, DRF, SQLite/MySQL, Redis, WebSockets.

Main modules: Student Tracking Portal, Agent Mobile Action Panel, Admin Management Dashboard.

Main database models: Application, Agent, AgentAssignment.

Agent workflow: Receive task → Visit university → Log proof → Upload final transcript.

Current limitations: Payment gateway is not fully integrated. File storage is local (needs S3).

Production readiness: Partially ready. Requires S3 migration, MySQL database provisioning, and a production ASGI server setup (Daphne).

Recommended next steps:

1. Integrate Stripe/Razorpay.
2. Configure AWS S3 bucket for media.
3. Deploy to a managed service (e.g., AWS EC2, DigitalOcean, or Heroku).